

CSCE 713 SOFTWARE SECURITY · MODULE 11 · LECTURE NOTES

Code Review for Security

The only method that finds design and logic defects.

MODULE OUTCOMES

- Choose what to review, by risk.
- Apply a systematic reading method.
- Explain what review finds that tooling cannot.
- Review a diff for security specifically.
- Write a finding that gets fixed.

1 Choosing what to review

- **Anything that parses untrusted input** — the highest concentration of both memory and logic defects in any codebase, and the place where Modules 02 through 06 all apply at once.
- **Authentication and authorisation code**, and every place that decides whether an action is permitted (CSCE 701 Module 03) — because the tooling finds none of this (§3's table), so review is not a second opinion here but the only opinion.
- **Anything that runs with elevated privilege**, or that crosses a trust boundary (Module 06 §4) — where the consequence of a defect is multiplied by the context (Module 01 §3).
- **Cryptographic use**, against CSCE 711 Module 11's five mechanical checks — a non-constant-time comparison, a caller-supplied nonce, a distinguishable error, a doubled-purpose key, and parsing before verification. **Five greps, and high yield.**
- **And the code nobody understands** — old, undocumented, written for a different context, and still reachable — which is where the assumptions have rotted (Module 01 §1's last cause). **'Parses untrusted input' and 'decides whether an action is permitted' are the two highest-yield categories**, and they fail in completely different ways, which is why a reviewer needs both halves of this course.

REVIEW WHAT TOOLING CANNOT SEE, AND LET TOOLING HAVE THE REST

A reviewer spending their attention on patterns a linter finds is wasting the scarcest resource in the entire process — and will be measurably less careful by the time they reach the part that actually needed a person, because attention is finite within a single review.

So automate the mechanical checks (Module 07) and spend the review on the three things only a person can assess:

Is the design right? Is the authorisation complete? And does this code do what it claims to do? — none of which a tool can evaluate, because none of the three is expressible without knowing the intent, and intent is not in the source.

Which makes review the *complement* to tooling rather than its backstop — and is precisely why Project 2 requires review alongside the automated techniques rather than as a final pass over whatever they flagged.

2 Reading

1. UNDERSTAND THE INTENDED BEHAVIOUR FIRST
you cannot spot a deviation from a specification you do not have. If none exists, reconstruct one from the tests and the callers before reading further.
2. FIND THE ENTRY POINTS
and rank them by who can reach them (pre-auth first -- Module 01 section 3)
3. FOR EACH, TRACE TO THE DANGEROUS OPERATIONS
allocation, indexing, copying, interpretation by something downstream, and privilege decisions (Module 01 section 2's four-step method)
4. AT EVERY STEP, ASK "WHAT IS ASSUMED HERE"
and then whether it is guaranteed on THIS path
5. THEN READ THE ERROR PATHS SPECIFICALLY
they are less tested, less reviewed, and they free things, skip checks, and return early
6. AND READ THE DIFF'S CONTEXT, NOT JUST THE DIFF
a locally correct change can break a caller's assumption that is not visible in the diff

Step 5 is the highest-yield single habit in this module. Error paths hold a disproportionate share of defects because essentially nothing exercises them — they are rarely unit tested, rarely hit in integration, almost never fuzzed into, and they perform exactly the operations that go wrong: freeing partially constructed objects, returning before a check, leaving a lock held, logging the thing that should not be logged. **Reading only the error paths of a component is a productive half-hour.**

- **'What happens if this is zero, negative, empty, maximal, or absent?'** — five cases, and one of them is **usually unhandled**. Mechanical, and it finds a great deal.
- **'Who checked this, and can this function be called without that check?'** — which is **Module 01 §2's step 4**, and the answer requires looking at the call graph rather than at the function.
- **'What happens if this fails?'** — and specifically **whether the caller can distinguish failure from success**, since an ignored error return is how a great many security checks come to be skipped.
- **'Who owns this memory, and when is it freed?'** — asked of every pointer that crosses a function boundary (Module 02 §2's unclear-ownership pattern).
- **And 'is this check in the right place?'** — because **a check in the client is not a check** (CSCE 701 Module 03 §3's chokepoint argument), and a check in the wrong layer is a check an attacker simply does not execute. **The zero-negative-empty-maximal-absent question is worth asking of every numeric and every collection parameter you encounter.**

3 What only review finds

Defect kind	Static analysis	Fuzzing	Review
Memory safety	Partly (Module 07 §2)	Yes — with sanitisers (Module 09 §3)	Partly
Injection	Yes	Sometimes	Yes
Integer defects	Yes	Yes	Partly
Concurrency	Poorly	Poorly	Partly — and it is the best of the three (Module 05 §4)
Missing authorisation	No	No	Yes
Logic and design flaws	No	No	Yes

The last two rows are why review cannot be replaced — and they are not minor categories. Missing authorisation is among the most common severe findings in real assessments, and a design flaw can make a system insecure while every line of its code is individually correct. So a programme consisting entirely of tooling has a structural blind spot covering two of the six rows, and no amount of additional tooling closes it — which is this table's argument and the justification for the module.

4 Writing up a finding

A FINDING GETS FIXED WHEN IT IS SPECIFIC, REACHABLE, AND ACTIONABLE

The location, precisely — file and line, and the specific condition under which the defect occurs, **rather than 'input validation is weak in this module'**, which cannot be acted on and will not be.

The path from untrusted input, stated concretely — **because a finding whose reachability is unestablished will be disputed and deferred**, and **that is the correct response to it** (Module 01 §1's three requirements). Establishing reachability is your job, not the maintainer's.

The impact you actually established, in Module 01 §3's honest form — **not the worst case you can imagine**, which costs you credibility on the next finding and misdirects the triage.

And a suggested fix, ideally **at the level that prevents recurrence** (Module 10 §2's interface change) **rather than only this instance** — which is what turns a finding into a durable improvement and is what Project 1's three-level fix exercise was training.

- **Review the change, not the author** — which is CSCE 701 Module 10 §4's blamelessness applied before the incident rather than after it, and it has the same justification: a blaming review process produces defensive authors and smaller, less honest changes.
- **Ask questions rather than assert defects when you are unsure**, because **you are reading without the context the author has** — and a question costs nothing while a wrong assertion costs credibility you will need later.

- **And accept that you will miss things. Review finds a fraction of what is present** — the measured rates are sobering — and **a programme that treats it as a guarantee has misplaced its confidence** (Module 13).
- **Record what you looked for**, so that the next reviewer does not repeat the same pass and so that **the claim about what was reviewed is checkable** by somebody else.
- **Which is what makes a review an assessment rather than an opinion. Recording what you looked for is the practice that turns review into evidence** — and it is exactly what Module 13 §1 asks you to report, and what Project 2 grades.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Dowd, McDonald & Schuh, chapters 1 through 4	§1 and §2's method — the most complete treatment of adversarial code reading in print. Library copy.
Google's code review developer guide (free)	§4's culture , from a very large practice — not security-specific, and the right foundation for it.
OWASP Code Review Guide (free)	§1 and §2 as a checklist — useful for completeness, and no substitute for §2's method.
Mozilla and Chromium security review notes (free)	§1's prioritisation in practice — how large projects decide what gets a security review at all.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Rank five components of your own project** by Part 1's yield order.
- 2 **List the mechanical checks** you should automate rather than review.
- 3 **Reconstruct the intended behaviour** of an undocumented component.
- 4 **Apply Part 2's six steps** to one parser, and record what you found.
- 5 **Read only the error paths** of one component, and report the yield.
- 6 **Ask the five-case question** of every parameter in one module.
- 7 **Find one check that can be bypassed** by a different call path.
- 8 **Find one check in the client** that is not enforced on the server.
- 9 **Write up one finding** in Part 4's form.

- 10 **Record what you looked for and did not find**, for Project 2.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give the five review priorities and the two highest-yield categories.
- 2 Why should a reviewer not check what a linter checks?
- 3 What three things can only a person assess?
- 4 Give Part 2's six steps, and say which habit yields most.
- 5 Why are error paths disproportionately defective?
- 6 Give the five questions that find real defects.
- 7 Why is a check in the client not a check?
- 8 Which defect kinds are found only by review?
- 9 Give the four components of a finding that gets fixed.
- 10 Why record what you looked for?